Computational Physics

Lecture 9: Monte Carlo methods

Monte Carlo integration

- hit-or-miss-method
- mean value method
- integrals in many dimensions
- importance sampling

Monte Carlo simulations and statistical mechanics

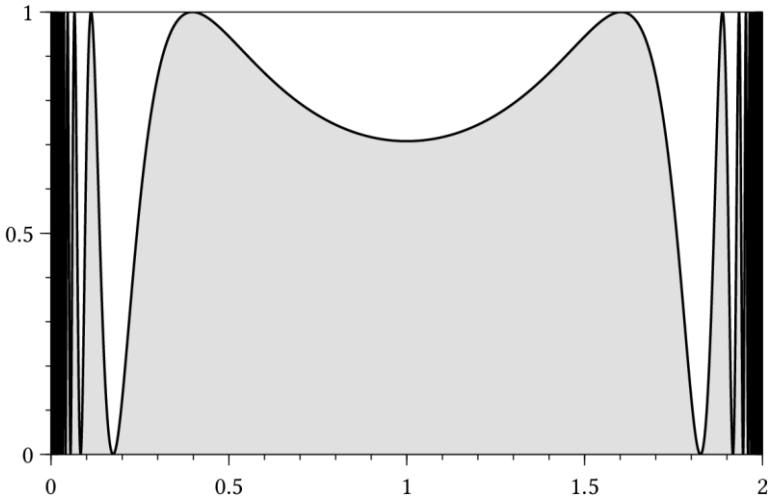
- Markov chains
- Metropolis algorithm
- Simulated annealing

Reference: Chapter 10 of *Computational Physics* by Mark Newman

Monte Carlo integration

Idea: start from an exact problem, such as the calculation of an integral, and find an approximate solution by running a suitable random process

Consider $$I = \int_0^2 \sin^2\left[\frac{1}{x(2-x)}\right] dx.$$



the function is well behaved in the center but varies infinitely fast at the edges. Makes the integral challenging using basic methods

We know the area *A=2* of the bounding rectangle

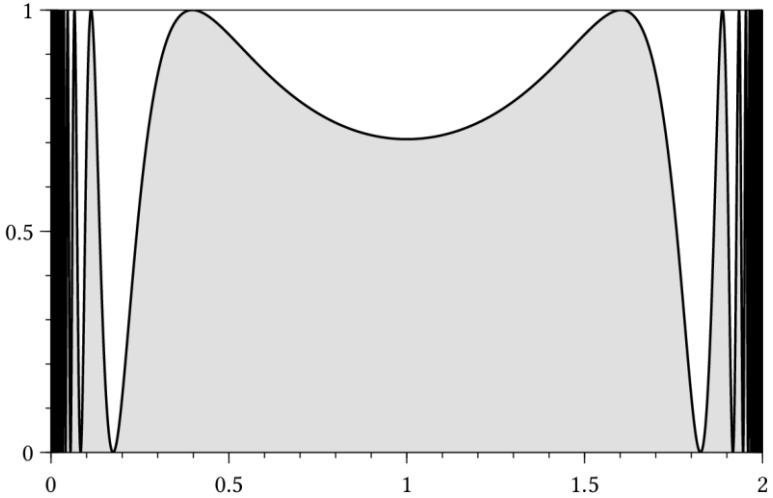
If we throw points at random in the rectangle, the probability that they fall in the shaded region is

$$p = \frac{I}{A}$$ connected to the area *I*!

Monte Carlo integration

Scheme:
1) generate a large number of N random points with uniform probability in the bounding rectangle
2) check points that fall below the curve and count their number *k*

Then $p \approx \frac{k}{N} \approx \frac{I}{A} \rightarrow$ $I \approx \frac{kA}{N}$



This technique is the **Monte Carlo integration** (from the casino in Monte Carlo)

Useful for pathological integrals, or multiple integrals in high dimensions

Monte Carlo integration

How to measure accuracy?

Given that points can fall below curve with probability *p* and above with probability *1-p*,
the probability that *k* of *N* points fall below the curve is the binomial distribution

$$P(k) = \binom{N}{k} p^k (1-p)^{N-k}. \qquad \text{with variance} \qquad \operatorname{var} k = Np(1-p) = N\frac{I}{A}\left(1-\frac{I}{A}\right)$$

Now that we have an estimate of the variation of k, the expected error on the integral *I* is

$$\sigma = \sqrt{\operatorname{var} k}\,\frac{A}{N} = \frac{\sqrt{I(A-I)}}{\sqrt{N}}.$$

The error scales with $N^{-1/2}$, so rather slowly: if we increase samples by 100 we reduce error by a factor 10.
by comparison for trapezoidal rule the error reduction was by a factor 10000

Note:

- *f*(*x*) cannot be negative (wrong handling of negative area). Would need to shift curve so that it is positive and enclosed, then subtract constant at end
- need to know function maximum value

Example of Monte Carlo integration

Implement the integral *I* and its error using the Monte Carlo integration

$$I = \int_0^2 \sin^2\left[\frac{1}{x(2-x)}\right] dx.$$

```
def f(x):
    return (np.sin(1/(x*(2-x))))**2

# Parameters
N = 10000
A = 2.0

# Separate arrays for points
x_in, y_in = [], []
x_out, y_out = [], []
count = 0

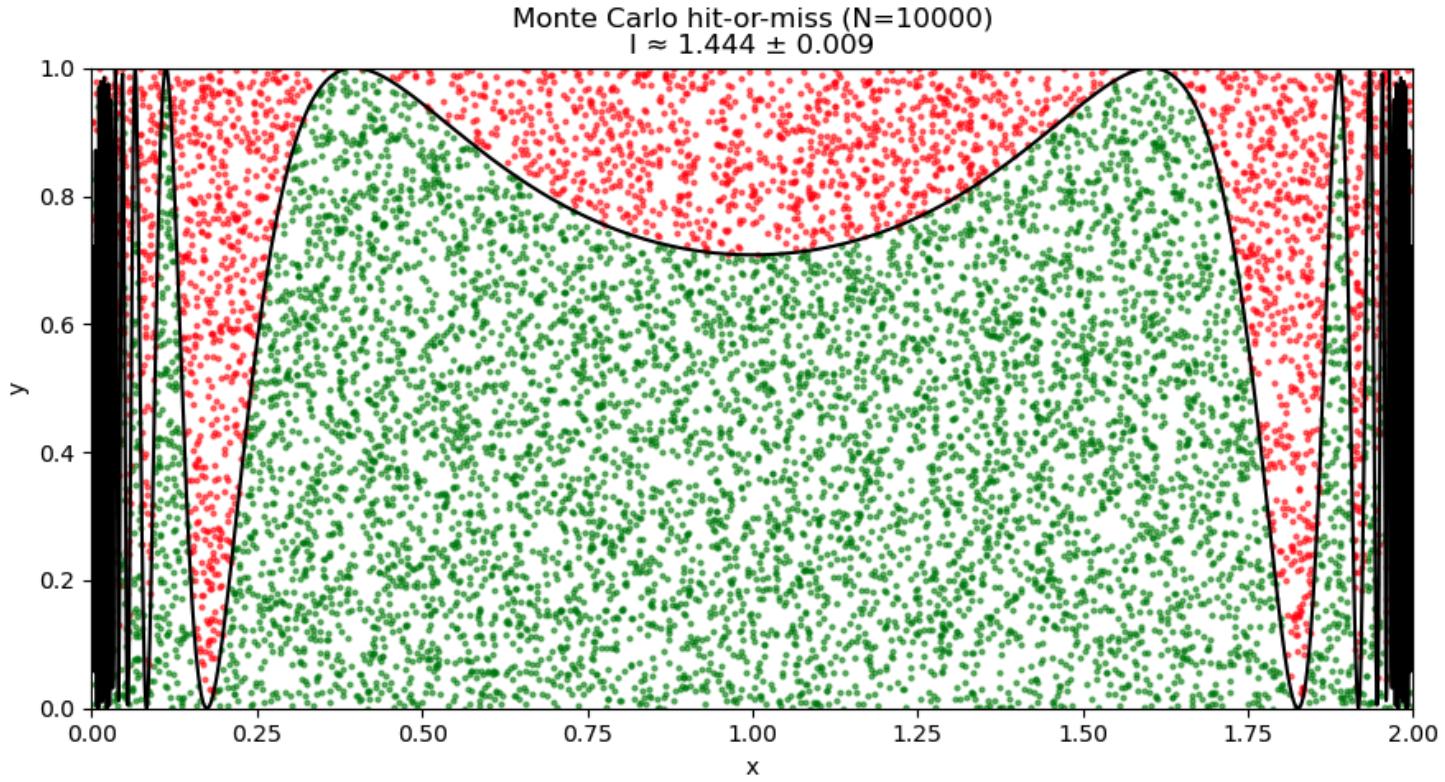
for i in range(N):
    x = 2 * np.random.rand()
    y = random()
    if y < f(x):
        x_in.append(x)
        y_in.append(y)
        count += 1
    else:
        x_out.append(x)
        y_out.append(y)

I = A * count / N
sigma = np.sqrt(I * (A - I)) / np.sqrt(N)

print(f"I ≈ {I:.4f} ± {sigma:.4f}")

# Function curve
x_curve = np.linspace(0.001, 1.999, 2000)
y_curve = [f(x) for x in x_curve]

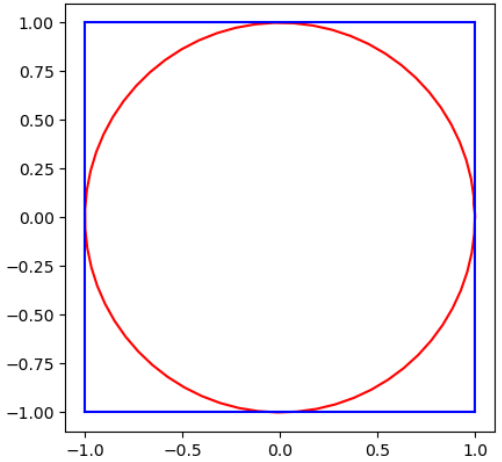
# Plot
plt.figure(figsize=(9, 5))
plt.plot(x_curve, y_curve, 'k-', lw=1.5, label=r"$f(x)$")
plt.scatter(x_in,  y_in,  s=4, c='green', alpha=0.5, label="below")
plt.scatter(x_out, y_out, s=4, c='red',   alpha=0.5, label="above")
plt.xlim(0, 2)
plt.ylim(0, 1)
```



Example of computing π with Monte Carlo

Consider a circle of unit radius r = 1. Its area is: $A = \pi r^2 = \pi$

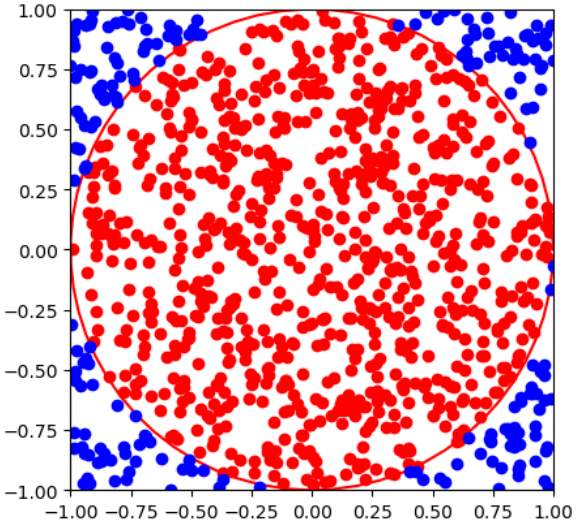
The circle can be embedded into a square with a side length of two and area A_{sq} = 4



If you now throw a uniform random point in the square, the probability to fall in the circle is $p = \frac{A}{A_{sq}} = \frac{\pi}{4}$

This probability can be estimated by sampling *N* points inside the square many times and counting how many fall *k* inside the circle. π can therefore be estimated as:

$$\pi \approx 4\frac{k}{N}$$



Mean value method: computing integral as an average

We can derive another method to compute an integral in [a,b]: $I = \int_a^b f(x)dx$

The mean value of *f(x)* over [a,b] is defined as: $\langle f \rangle = \frac{\int_a^b f(x)dx}{b-a} = \frac{I}{b-a}$,

Thus $I = (b-a)\langle f \rangle$ if we estimate $\langle f \rangle$ we can estimate I

To compute $\langle f \rangle$, sample $f(x)$ at N random points $x_1, \ldots x_N$ chosen uniformly in [a,b]:

$$\langle f \rangle = \frac{1}{N}\sum_{i=1}^{N} f(x_i). \longrightarrow \boxed{I \approx \frac{b-a}{N}\sum_{i=1}^{N} f(x_i).}$$

Mean value method: computing integral as an average

To estimate error, estimate variance as

$$\mathrm{var} f = \langle f^2 \rangle - \langle f \rangle^2 \qquad \text{with} \qquad \langle f \rangle = \frac{1}{N} \sum_{i=1}^{N} f(x_i) \qquad \text{and} \qquad \langle f^2 \rangle = \frac{1}{N} \sum_{i=1}^{N} [f(x_i)]^2$$

Then variance of $\sum_{i=1}^{N} f(x_i)$ is N var f

The error on integral is $\sigma = \frac{b-a}{N} \sqrt{N \,\mathrm{var}\, f} = (b-a) \frac{\sqrt{\mathrm{var}\, f}}{\sqrt{N}}.$

Same scaling as before, but you can prove it is more accurate because leading constant is smaller than hit-or-miss method

Note:

- the method works also if $f(x)$ is negative
- no need to know its maximum value

```
def intMC(f, N, a, b):
    total = 0
    total_sq = 0
    for i in range(N):
        x = a + (b-a)*np.random.rand()
        fval = f(x)
        total += fval
        total_sq += fval * fval
    f_av = total / N
    fsq_av = total_sq / N
    return (b-a) * f_av, (b-a) * np.sqrt((fsq_av - f_av*f_av)/N)
```

Integrals in many dimensions

Monte Carlo methods really shine when it comes to numerical evaluation of integrals in multiple dimensions. Consider the following d-dimensional integral:

$$I = \int_{a_1}^{b_1} dx_1 \int_{a_2}^{b_2} dx_2 \cdots \int_{a_d}^{b_d} dx_d \, f(x_1, x_2, \ldots, x_d)$$

Computing it numerically using e.g. the trapezoidal rule would involve the evaluation of a multi-dimensional sum.

Suppose we divide each interval in N points, it means evaluating f on a **grid with a number of points which scales exponentially** as N^d

After 4-dimensions it becomes impractical

Integrals in many dimensions

We can extend the mean-value method directly in d-dimensions $$I \simeq \frac{V}{N}\sum_{i=1}^{N} f(\mathbf{x_i})$$

where now $\mathbf{x}_i$ are independent random points picked uniformly at random within the volume V

The error also generalize to $$\sigma = V\frac{\sqrt{\mathrm{var}\, f}}{\sqrt{N}}$$

The error scales in the same way $O\left(N^{-1/2}\right)$, independently of d, while for the trapezoidal rule it is $O(h^2) = O\left(N^{-2/d}\right)$

spacing $h = \frac{L}{n} = \frac{L}{N^{1/d}}$ (L length of edge and n number of point along one direction)

For d<4 the error scales better for trapezoidal rule
For d=4 the errors are the same
For d>4 the errors scales faster for Monte Carlo

Importance sampling

When computing the integral as statistical average $I \approx \frac{b-a}{N} \sum_{i=1}^{N} f(x_i).$

we are having several inefficiencies:

- Uniform sampling: many points land in regions where f(x) ≈ 0
- no adaptive choice of sampling points when f varies strongly

Importance sampling:

Sample x_i from a *non-uniform* distribution *w(x)* that resembles *f(x)*

Tricky part is computing the mean value correctly from nonuniform points

Importance sampling

Start from any probability *w(x)* distribution to use for sampling *x*, correctly normalized $\int_a^b w(x)dx = 1.$

Define $g(x) = \frac{f(x)}{w(x)}$ so that we can write

$$\int_a^b f(x)\,dx = \int_a^b w(x)g(x)\,dx$$

we are expressing the integral as a **weighted average** of the function *g(x)* with weights *w(x)*

Suppose now I extract *N* sampling points x_i using w(x): I will get more points where w(x) is larger and viceversa (nonuniform samples). Thus

$$\int_a^b w(x)g(x)\,dx \simeq \frac{1}{N}\sum_{i=1}^{N} g(x_i)$$

The average over g(x_i) automatically coincides with the weighted average, because where w(x) is large, I will automatically sample more points x_i (and thus have more g(x_i) in the summation) and viceversa

Importance sampling

Just some simple example: take a discrete distribution with 3 values

x	$w(x)$	$g(x)$
1	0.1	10
2	0.2	20
3	0.7	30

with weighted average

$$0.1 \cdot 10 + 0.2 \cdot 20 + 0.7 \cdot 30 = 26$$

If I now decide to sample a large number of points x_i using w(x), at the end I will get

~10% points with $x = 1$
~20% points with $x = 2$
~70% points with $x = 3$

$$\sum_{i=1}^{N} x_i = 26$$

the sum will coincide to the weighted average

Importance sampling

Using again $g(x) = \frac{f(x)}{w(x)}$ we arrive at $\int_a^b f(x)\,dx \simeq \frac{1}{N}\sum_{i=1}^{N}\frac{f(x_i)}{w(x_i)}$

if we sample the points nonuniformly with *w(x)* and average $f(x_i)/w(x_i)$ we get an estimation of our integral. This formula allows to choose a *w(x)* that sample in a preferential way, connected to *f(x)*

It may appear that we could divide by zero where w(x) is zero, but it never happens, as these points will never be draw if the probability is zero.

Another way of writing the same formula would be $I = \int_a^b \frac{f(x)}{w(x)} w(x)dx = \left\langle \frac{f(x)}{w(x)} \right\rangle_w$

Error $\sigma = \frac{\sqrt{\mathrm{var}_w(f/w)}}{\sqrt{N}}$, with $\mathrm{var}_w(f/w) \simeq \frac{1}{N}\sum_{i=1}^{N}\left[\frac{f(x_i)}{w(x_i)}\right]^2 - \left[\frac{1}{N}\sum_{i=1}^{N}\frac{f(x_i)}{w(x_i)}\right]^2$

Importance sampling

Importance sampling offers a solution also for doing integrals over infinite domains instead of doing a change in variables.

Although we cannot extract random points uniformly distributed in an infinite domain, we can use nonuniform distributions that extend to infinity but where probability becomes zero, e.g. an exponential distribution

$$\int_0^\infty f(x)\,dx \simeq \frac{1}{N}\sum_{i=1}^{N} e^{x_i} f(x_i), \qquad \text{with} \qquad w(x) = e^{-x}$$

Importance sampling also extends naturally to higher dimensions:

$$\int_V f(\mathbf{r})\,d^d\mathbf{r} \simeq \frac{1}{N}\sum_{i=1}^{N} \frac{f(\mathbf{r}_i)}{w(\mathbf{r}_i)},$$

Example using importance sampling

Integrate $I = \int_{-\infty}^{\infty} e^{-x^4}\, dx$ using a suitable distribution w(x), e.g. a Gaussian with mean zero and variance 1

This choice is suitable for the form of the distribution, allowing to integrate in the infinite domain

```
from math import exp,log,cos,sqrt,pi
from random import random

N = 1000000

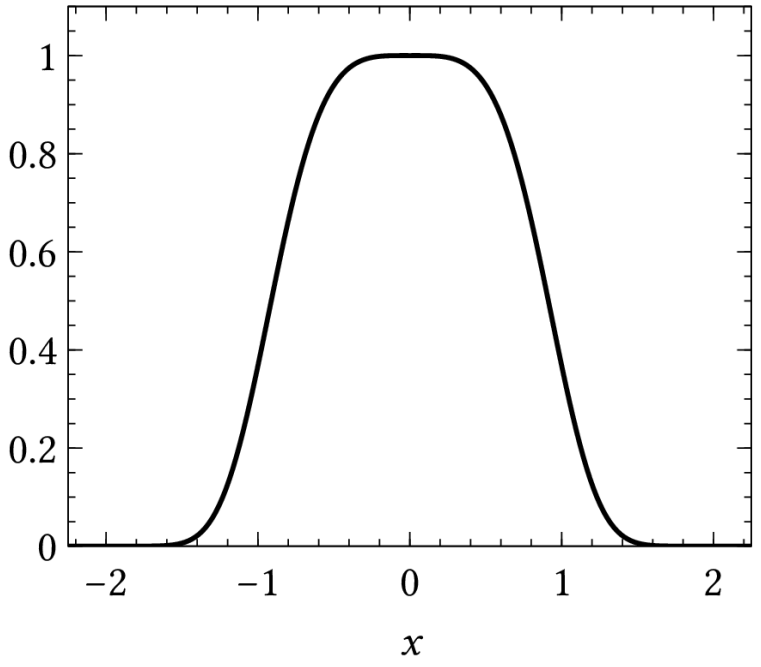
# The integrand
def f(x):
    return exp(-x**4)

# The Gaussian distribution with sigma=1
def p(x):
    return exp(-x**2/2)/sqrt(2*pi)

# Function to generate a Gaussian random number with mean 0
# and variance 1
def gaussian():
    r = sqrt(-2*log(1-random()))
    theta = 2*pi*random()
    return r*cos(theta)

s = 0.0
for k in range(N):
    x = gaussian()
    s += f(x)/p(x)

print("I =",s/N)
```



Monte Carlo simulations

Monte Carlo simulations is a name given to computer calculations that use random numbers to simulate a random physical process.

The radioactive decay or the random walk were an example of it.

Methods used in every branch of physics, but particularly often in statistical mechanics

Monte Carlo simulations and statistical mechanics

Recall that the fundamental problem of equilibrium statistical mechanics is to calculate the average value of a quantity of interest. A system at temperature T passes through a succession of states s, each with probability

$$P(s) = \frac{e^{-\beta E(s)}}{Z}$$

probability of occupying a certain state s

with: $\beta = \frac{1}{k_B T}$

$E(s)$ energy of state s

$Z = \sum_s e^{-\beta E(s)}$ **partition function**: sum of all probability weights for all states s, and is the normalization factor of *P(s)*

Main interest typically lies in calculating the average of various physical observables. For an arbitrary quantity *X(s):*

$$\langle X \rangle = \sum_s P(s)\, X(s)$$

Monte Carlo simulations and statistical mechanics

How to calculate $\langle X \rangle$?

Only in a few systems the sum can be done exactly, but mostly cannot be done due to huge sum over states

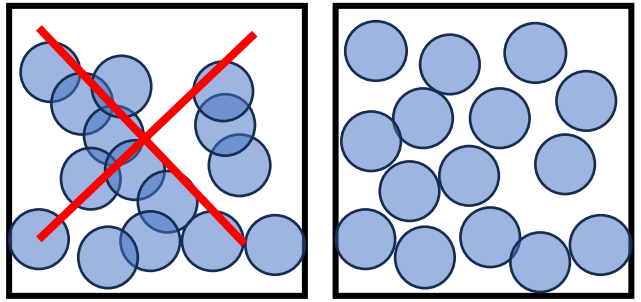
We could take the same approach as Monte Carlo integration: choose a set of *N* random s_i states and perform a weighted average using $e^{-\beta E(s_i)}$:

$$\langle X \rangle \approx \frac{\sum_{i=1}^{N} e^{-\beta E(s_i)} X(s_i)}{\sum_{i=1}^{N} e^{-\beta E(s_i)}}$$

Note that we need now to divide by the weight sum to have a correct normalization. This method does not require knowing *Z*

Monte Carlo simulations and statistical mechanics

Unfortunately, it doesn't work.



The Boltzmann probability is exponentially small in most cases, due to most states having high energy ($E \gg k_B T$ or $e^{-\beta E} \ll 1$, think e.g. about configurations with overlapping particles, which are more probable to be found but have zero Boltzmann weight due to having high interaction energy)

So typically the random states s_i we pick will make very little contribution to the original sum, and the average will be a poor estimation

To get around the problem: **Importance sampling.** Sample the microstates directly from the Boltzmann distribution

In importance sampling we draw configurations from nonuniform distribution to better sample the regions we are interested instead of a uniform one. Here we have to find a nonuniform distribution to draw the configurations that make the most significant contributions

Monte Carlo simulations and statistical mechanics

Let's say we use the probability $w(s)$ to sample states *s*, with $\sum_s w(s) = 1$

Then our average over states s can be expressed as:

$$\langle X \rangle = \sum_s P(s)\, X(s) = \sum_s w(s) \frac{P(s)\, X(s)}{w(s)} = \sum_s w(s)\, g(s)$$

a new average using the probability $w(s)$ and over the new quantity $g(s) = \frac{P(s)\, X(s)}{w(s)}$

If we do an importance sampling of *N* states s_i using $w(s)$ we will have that

$$\langle X \rangle \simeq \frac{1}{N} \sum_{i=1}^{N} g(s_i) = \frac{1}{N} \sum_{i=1}^{N} \frac{P(s_i)\, X(s_i)}{w(s_i)}$$

We would like to choose $w(s)$ that follows same peak and valleys as $P(s)$, so **let's choose** $w(s) = P(s)$:

$$\langle X \rangle \approx \frac{1}{N} \sum_{i=1}^{N} X(s_i)$$

Monte Carlo simulations and statistical mechanics

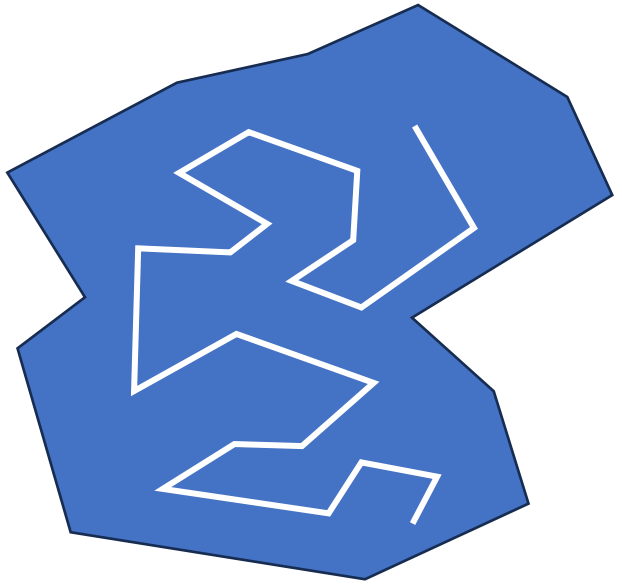
So **if we are able to sample our points using directly the Boltzmann distribution, we can compute any average directly as the average over sampled points**

$$\langle X \rangle \approx \frac{1}{N} \sum_{i=1}^{N} X(s_i)$$

The answer will be much more accurate than using uniform sampling.

How to sample directly with $P(s)$? A physical way would be to think as a real **system passing dynamically through a succession of states**, following the correct probability $P(s)$, in a simulation

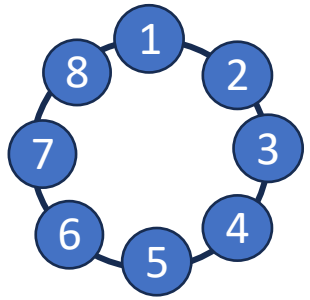
When we look at the phase space, we can think that only certain regions (not always connected) has a high probability. So, a Monte Carlo simulation means moving in a smart fashion in this space using $P(s)$ as your information.



A way to do this simulation is the **Markov chain method**.

Stochastic process and Markov chain

A stochastic process is a collection of random variables, typically in time



Let's start with this miniature Monopoly game. A player, starting from 1, throw a dice and moves in a clockwise direction.
The random variable X_n here is the position of the player at turn *n*.

Suppose you have the following game realization after 4 turns:

$$(X_0, X_1, X_2, X_3) = (1,3,4,7)$$

The question is: can I predict from this time series the position of the next move?
We can predict by hand that X_4 depends only on the last position $X_3 = 7$ and that it can go with some probability in one of the following states, depending on the result of the dice throw $X_4 \in \{8,1,2,3,4,5\}$.
If the dice is fair, they are all equally probable.

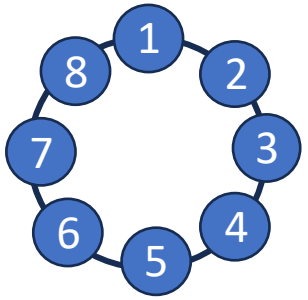
We can assign a probability to jumping to one of the available states conditioned to the precedent value:

$$P(X_4 = j | X_3 = 7) = \begin{cases} \frac{1}{6} \text{ for } j = 8,1,2,3,4,5 \\ 0 \text{ for } j = 6,7 \end{cases}$$

Stochastic process and Markov chain

Note that in principle the probability X_4 could depend on all the history, but for this specific process it only depends on the last realization:

$$P(X_4 = j | X_0 = 1, X_1 = 3, X_2 = 4, X_3 = 7) = P(X_4 = j | X_3 = 7)$$



With the property that each outcome depends only on the one that precedes it, the process is called **Markov chain**

Note that I can write for each initial state *i* the probabilities to transition to all other states j

$$P(X_{n+1} = j | X_n = i)$$

Moreover, these conditional probabilities do not vary over time (it does not matter if I start in i=8 at time 0 or time n), making the Markov chain **time-homogeneous**

$$P(X_{n+1} = j | X_n = i) = P(X_1 = j | X_0 = i)$$

Stochastic process and Markov chain

Finally I can construct a matrix with all these probabilities $P(X_1 = j | X_0 = i)$, called **transition matrix** T_{ji} (note we use the convention where end point is first index)

T_{ji} — start point *i* →; ↓ end point *j*

	1	2	3	4	5	6	7	8
1	0	0	1/6	1/6	1/6	1/6	1/6	1/6
2	1/6	0	0	1/6	1/6	1/6	1/6	1/6
3	1/6	1/6	0	0	1/6	1/6	1/6	1/6
4	1/6	1/6	1/6	0	0	1/6	1/6	1/6
5	1/6	1/6	1/6	1/6	0	0	1/6	1/6
6	1/6	1/6	1/6	1/6	1/6	0	0	1/6
7	1/6	1/6	1/6	1/6	1/6	1/6	0	0
8	0	1/6	1/6	1/6	1/6	1/6	1/6	0

notice how all columns sum up to 1

$$\sum_j T_{ji} = 1$$

probabilities from going to state *i* to any state *j* must sum up to 1

Stochastic process and Markov chain

Now we can look at the Markov chain in two equivalent and complementary ways:

- a random evolution in time *n* in which I get a **sequence of** X_n, where I draw a new value from the previous one using the conditional probability

- an **evolution of the probability distribution** $P(X_n = i)$ of finding state i at time *n*. This evolution is done in the following way: start from an initial distribution at time 0, e.g. in our case everyone starts from i=1

$$P(X_0 = 1) = 1 \quad \text{and} \quad P(X_0 = i) = 0 \text{ with } i = 2, \dots 8$$

the distribution at time 1 to be in state *j* will be given by

$$P(X_1 = j) = \sum_{i=1}^{8} P(X_1 = j|X_0 = i)P(X_0 = i) = P(X_1 = j|X_0 = 1)P(X_0 = 1) + P(X_1 = j|X_0 = 2)P(X_0 = 2) + \cdots$$

$$\boxed{P(X_1 = j) = \sum_i T_{ji}\ P(X_0 = i)}$$

This evolves directly the probability of finding all states at each step, and not just a random sequence of the states

Stochastic process and Markov chain

If $P(X_0 = i)$ is normalized, it will be normalized as well in the next step

$$\sum_i P(X_0 = i) = 1 \longrightarrow \sum_i P(X_1 = i) = 1$$

Defining the probability distribution vector at time 0 $\mathbf{P}^{(0)} = \begin{pmatrix} P(X_0 = 1) \\ P(X_0 = 2) \\ \dots \end{pmatrix}$

At the next step we have $\mathbf{P}^{(1)} = \mathbf{T}\mathbf{P}^{(0)}$

with **T** the transition matrix. At the n-th step $\mathbf{P}^{(n)} = \mathbf{T}^{\boldsymbol{n}}\mathbf{P}^{(0)}$

Stochastic process and Markov chain

We can ask if a **limiting distribution** exist, defined as

$$\mathbf{P} = \lim_{n\to\infty} \mathbf{T}^{\boldsymbol{n}}\mathbf{P}^{(0)}$$

a distribution of probabilities independent of the initial state, and which will not change when reached. In this sense P is also a **stationary distribution**, as it satisfies

$$\mathbf{P} = \mathbf{TP}$$

If one starts from a stationary distribution, it remains unchanged. Note that a stationary distribution might exist, but the evolution does not converge to it.

In our case P is known and T is the incognita: we want to design a time-homogeneous Markov chain process with a transition matrix **T**, such that the target probability **P** remains stationary.
And we know that this is obtained if T satisfies **P=TP**. Once determined T, we can use it to produce directly a random sequence of states with correct distribution.

Stochastic process and Markov chain

Let's write explicitly the equation that T needs to satisfy, **P**=**PT** for the general statistical mechanics system with target probability P(s)

$$P(r) = \sum_s T_{rs}\, P(s) \longrightarrow \boxed{\sum_s T_{sr}\, P(r) = \sum_s T_{rs}\, P(s)}$$

$$\sum_s T_{sr} = 1$$

global balance equation

needs to be valid for any state *r*

flux of total outgoing probability from r to other states s = flux of total incoming probability in r from all states s

If the two fluxes balance, P(r) is stationary

We can have however a more restrictive requirement: that the probability flux is balanced for each couple of states:

$$\boxed{T_{sr}P(r) = T_{rs}P(s)}$$

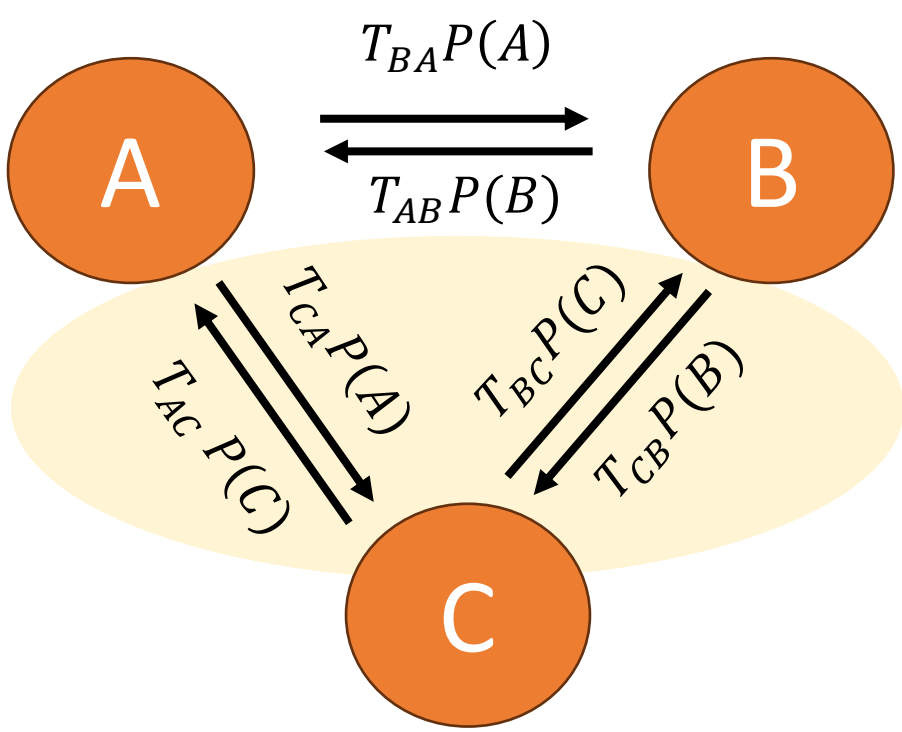
detailed balance equation

needs to be valid for any couple of states *r,s*

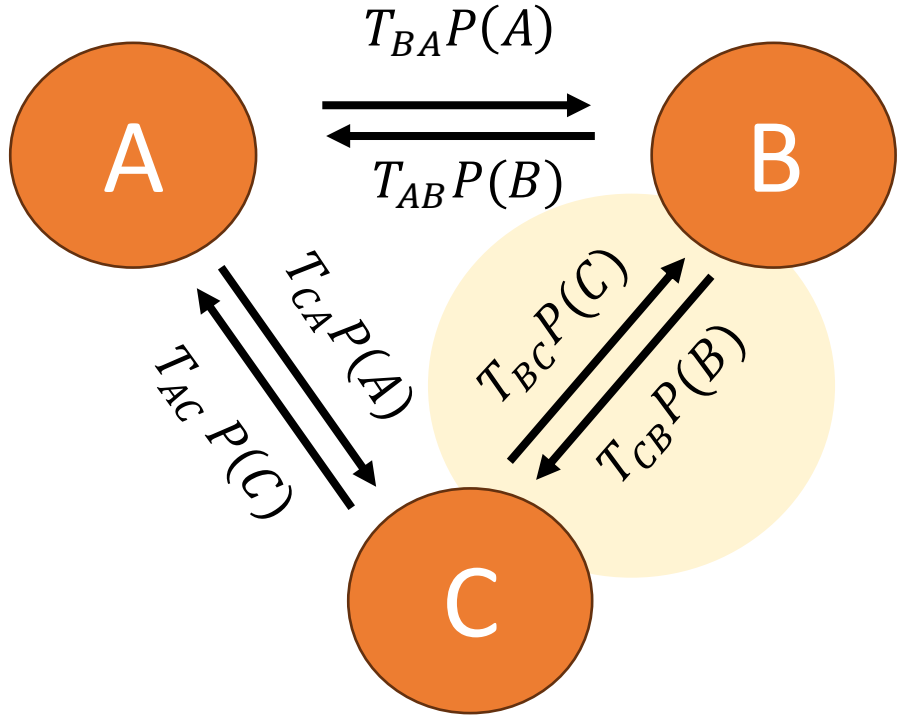
This is the typical requirement employed in Monte Carlo simulations.

Stochastic process and Markov chain

Consider a 3-state system and compare the requirements of the two methods



For global balance all outgoing and ingoing arrows (e.g in C) must be balanced for each state



For detailed balance all pairs of outgoing and ingoing arrows must be balanced for couple of state (e.g. B and C)

Metropolis algorithm (or Metropolis-Hastings algorithm)

Now we identify the transition matrix that has as target distribution $P(s) = \frac{e^{-\beta E(s)}}{Z}$

We want a T_{rs} that does not depend on *Z* and that satisfies detailed balance, in order to have a Markov chain process:

$$T_{rs}P(s) = T_{sr}P(r) \longrightarrow T_{rs}\frac{e^{-\beta E(s)}}{Z} = T_{sr}\frac{e^{-\beta E(r)}}{Z} \longrightarrow$$

$$\boxed{\frac{T_{rs}}{T_{sr}} = e^{-\beta(E(r)-E(s))}}$$

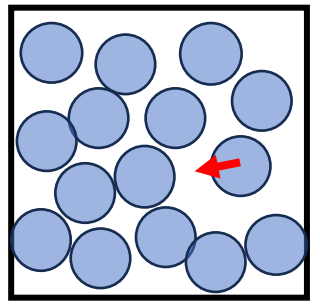
this equation just dictates conditions about the ratio, but still leaves room for the choice of T_{rs}.

Most common choice is **Metropolis algorithm**

Metropolis algorithm

The Metropolis algorithm involves splitting the transition probability into the product of two probabilities:

- make a **move** from a state s to a state r, **chosen uniformly from** a specified set of possibilities, called a **move set**. If there are M possibilities, the probability to choose r is 1/M. The move can be anything dependent on the system, e.g. taking a random particle and moving by a random quantity



- accept or reject the new state with **acceptance probability** P_a given by

$$P_a = \begin{cases} 1 & \text{if} \quad E(r) \leq E(s) \\ e^{-\beta(E(r)-E(s))} & \text{if} \quad E(r) > E(s) \end{cases}$$

if the move is rejected, we do nothing: the system remains in its old state s for one more step or in another words the move will be from state *s* to state *s*. If the move is accepted, the system moves to state *r*.

Another way to interpret it is that if we go towards lower energies we always accept the move, while if we move to higher energies, we may accept it with probability $e^{-\beta(E(r)-E(s))}$.

Metropolis algorithm

Let's verify that $\frac{T_{rs}}{T_{sr}} = e^{-\beta(E(r)-E(s))}$ is respected.

Consider that $T_{rs} = \frac{1}{M} P_a$ with $P_a = \begin{cases} 1 & \text{if } E(r) \leq E(s) \\ e^{-\beta(E(r)-E(s))} & \text{if } E(r) > E(s) \end{cases}$

If $E(r) > E(s)$ $T_{rs} = \frac{1}{M} e^{-\beta(E(r)-E(s))}$ and $T_{sr} = \frac{1}{M}$

If $E(r) \leq E(s)$ $T_{rs} = \frac{1}{M}$ and $T_{sr} = \frac{1}{M} e^{-\beta(E(s)-E(r))}$

In both instances, the ratio of the two quantities is $\frac{T_{rs}}{T_{sr}} = e^{-\beta(E(r)-E(s))}$

Metropolis algorithm

Steps of Markov chain Monte Carlo using Metropolis rule:

- Start from some initial state (chosen randomly, typically some low energy configuration in order to avoid a long equilibration process)
- Choose a move $s \rightarrow r$ from a uniform distribution of allowed moves.
- Compute $E(r) - E(s)$, then generate a random number z in [0,1), and accept the move if $z < e^{-\beta(E(r)-E(s))}$. If rejected, the system remains in its current state for one more step
- Measure the quantity of interest X in the current state and add it to a running sum, in order to later compute its average

Metropolis algorithm

Note that:

- If you reject a move and do not change the state, that new step with the old state still count towards averages. So if you stayed twice in the same state, the value of X in that state will be counted twice.

- It is crucial that the number of moves you choose for going from $s \to r$ is the same as going back from $r \to s$, otherwise it does not satisfy detailed balance. E.g. moving a particle randomly in a symmetric interval around 0 is ok, $\Delta x \in [-\frac{d}{2}, \frac{d}{2})$, but not moving asymmetrically in one direction $\Delta x \in [0, d)$

- the move choice should be as smart as possible, in order to be able to sample all possible states (ergodicity). Some choices might not allow to explore the entire phase space

- **Equilibration** from the initial state is important: the simulation must be longer than the time it takes to decorrelate from the initial state. This time gives also an idea about how many independent states you sampled in your simulation. Autocorrelation times of macroscopic quantities are a good proxy for computing such time.

Monte Carlo simulation of an ideal gas in a finite volume

Recall the energy states of a particle of mass m in a cubic box of length *L*

$$E(n_x, n_y, n_z) = \frac{\pi^2 \hbar^2}{2mL^2}(n_x^2 + n_y^2 + n_z^2) \qquad n_x, n_y, n_z = 1, 2, \ldots, \infty$$

An ideal gas of N such atoms that do not interact with each other has a total energy that is just the sum of the energies of each particle

$$E = \sum_{i=1}^{N} E\left(n_x^{(i)}, n_y^{(i)}, n_z^{(i)}\right)$$

Given that the probability of a single microstate of energy *E* is $P \propto e^{-\beta E}$

We can perform a Monte Carlo simulation to sample the energy of the gas at fixed temperature.

Monte Carlo simulation of an ideal gas in a finite volume

Move set:
1) pick a random particle
2) pick one of the 3 quantum numbers n_x n_y n_z
3) choose a random change for that quantum number, either +1 or -1

E.g, if we change n_x for atom i by +1, the change in total energy will be

$$\Delta E = \frac{\pi^2\hbar^2}{2mL^2}\left[(n_x+1)^2+n_y^2+n_z^2\right] - \frac{\pi^2\hbar^2}{2mL^2}(n_x^2+n_y^2+n_z^2) = \frac{\pi^2\hbar^2}{2mL^2}\left[(n_x+1)^2-n_x^2\right] = \frac{\pi^2\hbar^2}{2mL^2}(2n_x+1)$$

if the change is by -1 is

$$\Delta E = \frac{\pi^2\hbar^2}{2mL^2}(-2n_x+1)$$

Note that whenever n_x=1, if the change is -1 the move should always be rejected (cannot be less than 1)

Monte Carlo simulation of an ideal gas in a finite volume

Start from N=1000 atoms, L=10 and $k_B T = 10$. Set $m = \hbar = 1$.

Initialize the system with all particles in the ground state n_x=n_y=n_z=1. You can allocate an array of size (N,3) where you will save the state of each particle.

At each step, apply the random move, compute ΔE and accept or reject the move according to Metropolis rule. Perform 10^7 steps (it will require around a minute)

Monitor the value of E as a function of time, identifying the number of steps required to decorrelate from the initial conformation (which is not equilibrated)

Compare the value of E at equilibrium with the equipartition theorem value

Monte Carlo simulation of an ideal gas in a finite volume

```
T = 10.0
L = 10.0
N = 1000
steps = 10000000

# Create a 2D array to store the quantum numbers
n = np.ones((N, 3), dtype=int)

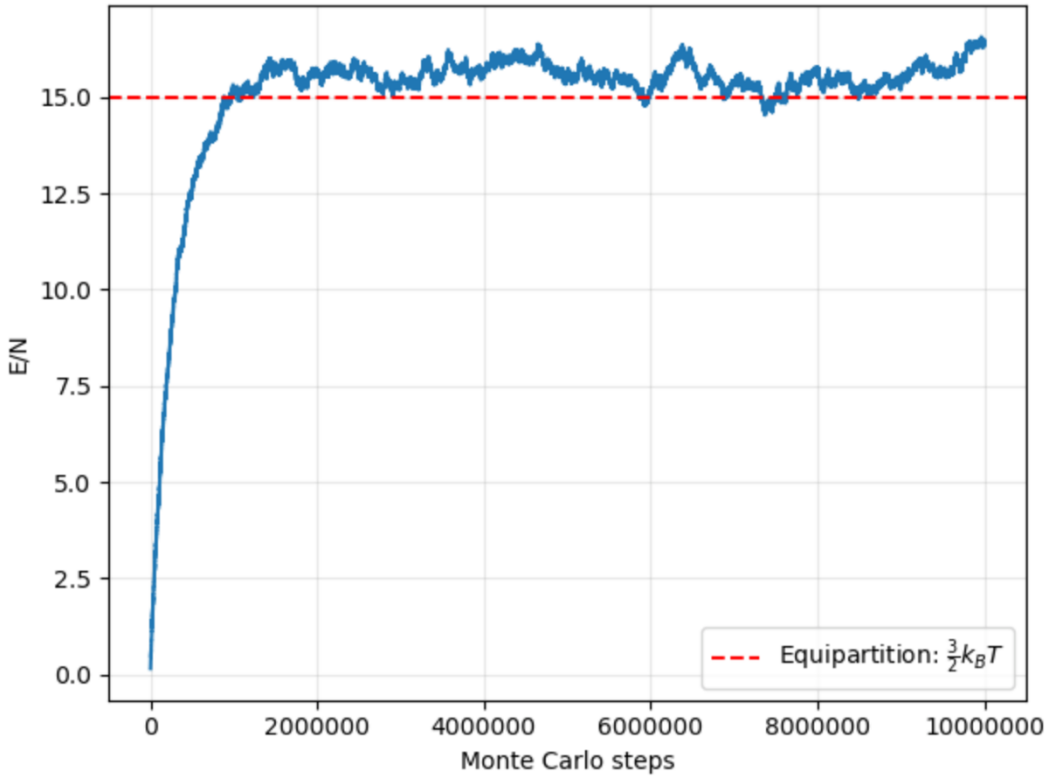
# Main loop
eplot = np.empty(steps)
E = 3 * N * np.pi**2 / (2 * L**2)

for k in range(steps):
    # Choose the particle and the move
    i = np.random.randint(N)
    j = np.random.randint(3)
    if np.random.random() < 0.5:
        dn = 1
        dE = (2 * n[i, j] + 1) * np.pi**2 / (2 * L**2)
    else:
        dn = -1
        dE = (-2 * n[i, j] + 1) * np.pi**2 / (2 * L**2)

    # Decide whether to accept the move
    if n[i, j] > 1 or dn == 1:
        if np.random.random() < np.exp(-dE / T):
            n[i, j] += dn
            E += dE

    # save energy per particle
    eplot[k] = E / N

# Make the graph
plt.plot(eplot)
plt.ylabel("E/N")
plt.ticklabel_format(style="plain")
plt.axhline(1.5 * T, color='r', linestyle='--',
            label=r'Equipartition: $\frac{3}{2} k_B T$')
plt.legend()
plt.xlabel("Monte Carlo steps")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

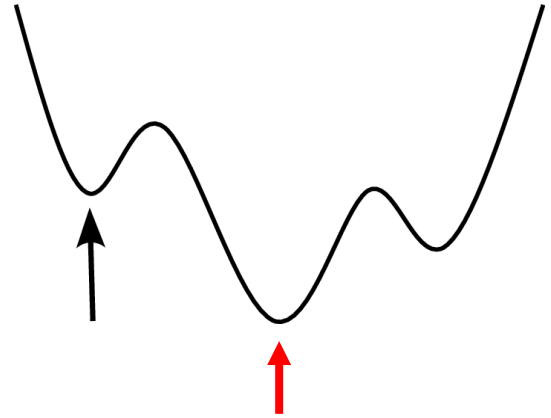


The number of steps needed to decorrelate from the initial state is about 1.5 million. Averages and distributions should be computed by sampling only after this point. This decorrelation time gives an order of magnitude of how many steps should separate successive samples if we want independent configurations.

Simulated annealing

Another important use of Monte Carlo simulations is in numerical optimization.

The methods for looking at maxima and minima typically get stuck into a local minimum, while sometimes **we want to find the global maximum or minimum** of a function



It applies to:

- finding the ground-state energy of a system
- finding the shortest delivery route between many locations (minimize fuel)
- circuit design: efficiently placing and wiring components
- job scheduling
- protein folding

Global optimization problems are among the hardest computational tasks. A most promising approach is simulated annealing from Scott Kirkpatrick

Simulated annealing

Start from a physical system at temperature T and Boltzmann probability of a state s: $P(s) = \frac{e^{-\beta E(s)}}{Z}$

Let's assume there is a single unique ground state, with reference energy *zero* such that *E(s)*>0 for all other states.

In the limit $T \to 0$:

- for the ground state $e^{-\beta E(s)} = e^0 = 1$
- for all other states the weight $e^{-\beta E(s)} \to 0$
- $Z = 1$

$$P(s) = \begin{cases} 1 & \text{for} \quad E(s) = 0 \\ 0 & \text{for} \quad E(s) > 0 \end{cases}$$

To find the ground state we can just cool down the system to T=0

In practice simulate a system at temperature T via Markov chain Monte Carlo and gradually lower the temperature.

Appliable to find the global minimum of any function $f(x, y, z, \ldots)$! The variable $x, y, z, \ldots$ would define the system state and *f* would represent your energy. Find also global maximum doing same process with *–f.*

Simulated annealing

How fast should we decrease T?

Same idea of annealing a piece of molten glass or hot metal:

Too fast, and the material develops defects, becoming soft or brittle. We are trapping the system in a local minimum energy, instead of the global one, because atoms didn't have time to rearrange themselves

With a **slow enough cooling**, instead, we **are able to reach the global minimum**

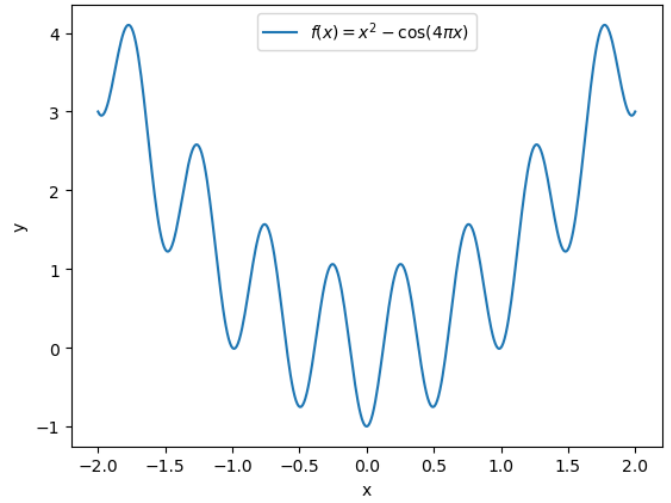
In practice:

- start with a high T_0, such that $\beta\big(E(s) - E(r)\big) \ll 1$ and $P_a \simeq 1$ and most moves are accepted
- specify a cooling schedule $T = T_0 e^{-t/\tau}$
- stop when system state does not change anymore

Need trial and error to find good value of relaxing constant τ, and is typically a compromise between relaxation being too slow and being too fast and missing global minimum

Global minimum of a function using simulated annealing

Find the global minimum (which is at x=0) of $f(x) = x^2 - \cos 4\pi x$



- Start e.g. from x=2
- at each step consider a move $x \to x_{cand} = x + \Delta x$ where Δx is drawn e.g. from the *normal distribution*
- Accept the move Markov chain Monte Carlo with Metropolis rule:

$$P_a = e^{-[f(x_{cand}) - f(x)]/T}$$

- Lower the temperature *T* at each step via the exponential *cooling schedule* $T_n = T_{max} e^{-n/\tau}$ and stop the process once *T* goes below a certain threshold T_{min}
- plot x as a function of the step to observe the annealing
- adjust T_{max}, T_{min} and τ until you observe good answer in reasonable time

Simulated annealing

```python
def f(x):
    return x**2 - np.cos(4 * np.pi * x)

x = 2.0
T_max = 3.0
T_min = 1e-4
tau = 100
sigma = 1

history = []
n = 0

while True:
    T = T_max * np.exp(-n / tau)
    if T < T_min:
        break

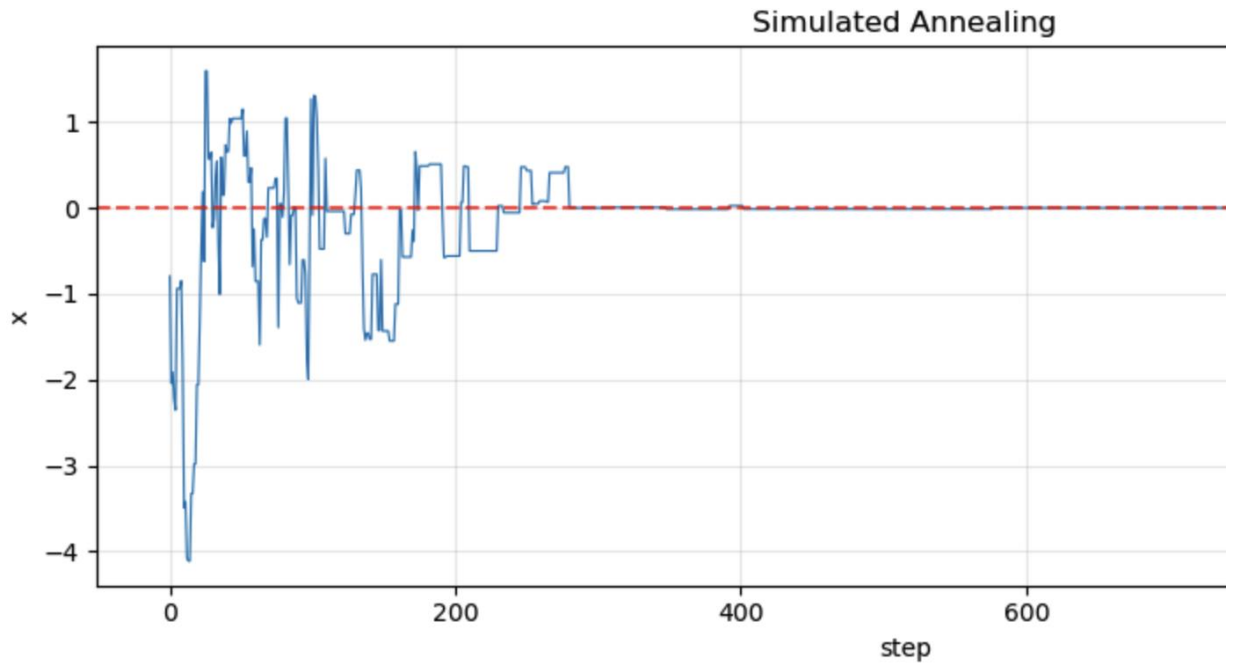
    x_cand = x + np.random.normal(0, sigma)
    delta = f(x_cand) - f(x)

    # Metropolis
    if delta < 0 or np.random.rand() < np.exp(-delta / T):
        x = x_cand

    history.append(x)
    n += 1

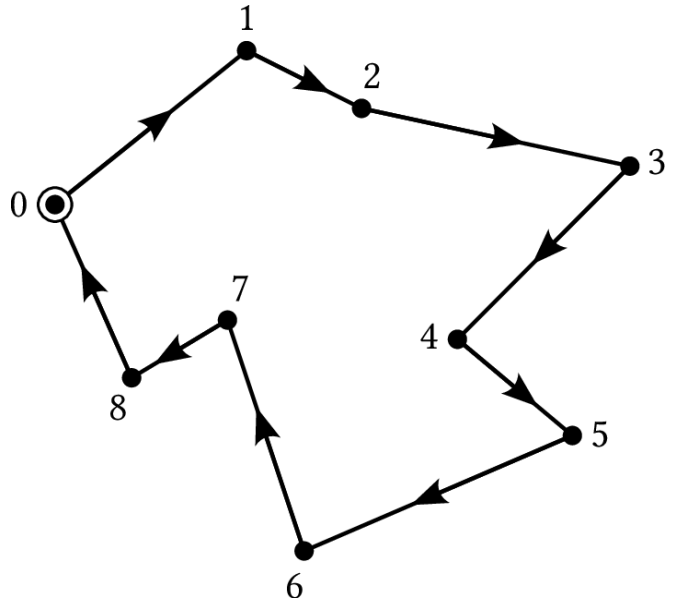
print(f"x final = {x:.6f}")
print(f"f(x)    = {f(x):.6f}")
print(f"step    = {n}")

# Plot
plt.figure(figsize=(10, 4))
plt.plot(history, lw=0.8)
plt.axhline(0, color="r", ls="--", alpha=0.7, label="global minimum x=0")
plt.xlabel("step")
plt.ylabel("x")
plt.title("Simulated Annealing")
plt.legend()
```



Simulated annealing and the traveling salesman problem

Traveling salesman problem:
You have *N* cities where each pair of cities is connected by straight roads.
The goal is to find **the shortest tour** in terms of total distance travelled, **that starts and ends at the same city and visits all *N* cities**.



It is a famous **NP-hard** problem: exact solution can be found via brute force O(N!) (check all permutations), or exponentially in a time $O(N^2 2^N)$ (Held-Karp algorithm)

Define each city position *i*: $\mathbf{r}_i = (x_i, y_i)$ with $\mathbf{r}_N = \mathbf{r}_0$

the total distance D traveled is $D = \sum_{i=0}^{N-1} |\mathbf{r}_{i+1} - \mathbf{r}_i|$

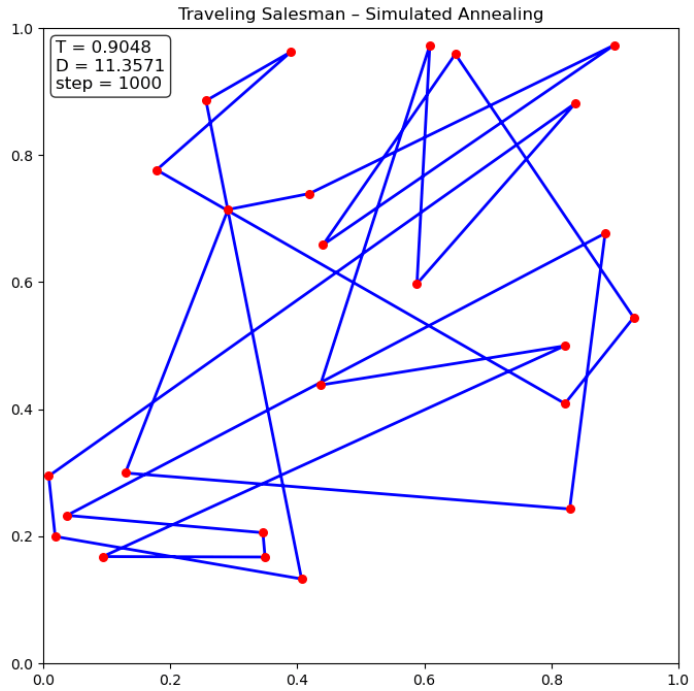
we need to find the order of the cities that minimizes *D* and annealing is perfect for the job

The move can be swapping a pair of cities

Simulated annealing and the traveling salesman problem

How to implement annealing:

- Start with random path
- At each step swap two cities from the path and compute the total distance change ΔD
- Accept the new path with probability $P_a = \exp(-\Delta D/T)$ (always accepted if ΔD<0)
- Decrease T in next step in accordance with annealing schedule



Simulated annealing and the traveling salesman problem

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# Parametri
N = 25
R = 0.03
Tmax = 1.0
Tmin = 1e-3
a = 1e-4

def mag(x):
    return np.sqrt(x[0]**2 + x[1]**2)

def distance(r):
    s = 0.0
    for i in range(N):
        s += mag(r[i+1] - r[i])
    return s

# Initialize city positions randomly
r = np.empty((N+1, 2), dtype=float)
r[:N] = np.random.random((N, 2))
r[N] = r[0]
D = distance(r)

# Setup gif
fig, ax = plt.subplots(figsize=(8, 8))
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.set_aspect('equal')
ax.set_title('Traveling Salesman — Simulated Annealing')

cities = ax.scatter(r[:N, 0], r[:N, 1], s=R*1000, c='red', zorder=5)
line, = ax.plot(r[:, 0], r[:, 1], 'b-', lw=2, zorder=4)
text = ax.text(0.02, 0.98, '', transform=ax.transAxes, va='top', fontsize=12,
               bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))
```

```python
frames = []
t = 0
T = Tmax
while T > Tmin:
    t += 1
    T = Tmax * np.exp(-a * t)

    #select cities to swap
    i, j = np.random.randint(1, N, size=2)
    while i == j:
        i, j = np.random.randint(1, N, size=2)

    #swap cities and compute deltaD
    oldD = D
    r[i], r[j] = r[j].copy(), r[i].copy()
    D = distance(r)
    deltaD = D - oldD

    #if move rejected, swap back cities
    if np.random.random() > np.exp(-deltaD / T):
        r[i], r[j] = r[j].copy(), r[i].copy()
        D = oldD

    if t % 1000 == 0:
        frames.append((r.copy(), T, D, t))

def update(frame_idx):
    r_frame, T_frame, D_frame, t_frame = frames[frame_idx]
    line.set_data(r_frame[:, 0], r_frame[:, 1])
    text.set_text(f'T = {T_frame:.4f}\nD = {D_frame:.4f}\nstep = {t_frame}')
    return line, text

ani = FuncAnimation(fig, update, frames=len(frames), interval=50, blit=False)
ani.save('tsp_sa.gif', writer='pillow', fps=20)
plt.close(fig)
print('Saved: tsp_sa.gif')
```